STEP 1: Importing relevant libraries

In [234]:
```
# Import relevant libraries
import pandas as pd
```

STEP 2: Loading the dataset

In [235]:
```
# Reading the supplements dataset
sales_data =
pd.read_excel("../EDA/data/dirty_supplement_sales_data_2020.xlsx"
)
```

STEP 3: Inspecting Data

In [236]:
```
# View the top 5 rows
print(f"Top 5 rows:  {display(sales_data.head())}")

# Viewing the last 5 rows
print(f"Last 5 rows: {display(sales_data.tail())}")
```

	Date	Product Category	Product Name	Units Sold	Units Returned	Price	Di
0	2020-10-07	Sleep Aids	ImmunoPro	NaN	1.0	$10.99	10
1	2020-09-16	Vitamins	MuscleMax	NaN	-1.0	NaN	0.(
2	2020-11-11	Immunity Boosters	Slim F@st	15.0	-1.0	15	Na
3	2020-11-04	Energy Drinks	PowerFuel	14.0	0.0	$10.99	25
4	2020-06-03	Vitamins	SuperV!ta	NaN	NaN	$10.99	5(

```
Top 5 rows:  None
```

	Date	Product Category	Product Name	Units Sold	Units Returned	Price
3095	2020-09-16	Immunity Boosters	SuperV!ta	10.0	2.0	NaN
3096	2020-08-12	Vitamins	DreamRest	25.0	0.0	NaN
3097	2020-10-21	Immunity Boosters	SuperV!ta	NaN	-1.0	$10.99
3098	2020-02-19	Weight Loss	SuperVita	NaN	NaN	NaN
3099	2020-08-26	Protein	PowerFuel	NaN	0.0	15

```
Last 5 rows: None
```

__Inspecting the shape of the dataset__

In [237]:
```
sales_data.shape
```

Out[237]:(3100, 10)

__Inspecting summary information about the dataset__

In [238]:
```
sales_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3100 entries, 0 to 3099
Data columns (total 10 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   Date              3100 non-null   object 
 1   Product Category  3100 non-null   object 
 2   Product Name      3100 non-null   object 
 3   Units Sold        1551 non-null   float64
 4   Units Returned    2495 non-null   float64
 5   Price             2457 non-null   object 
 6   Discount (%)      2651 non-null   float64
 7   Revenue           2457 non-null   float64
 8   Platform          3100 non-null   object 
 9   Location          3100 non-null   object 
dtypes: float64(4), object(6)
memory usage: 242.3+ KB
```

__Inspecting summary statistics__

Numerical Columns

In [239]:
```
sales_data.describe().T
```

Out[239]:

	count	mean	std	min	25%	50%
Units Sold	1551.0	24.514507	14.398598	1.0	11.5	24.0
Units Returned	2495.0	0.478557	1.120023	-1.0	-1.0	0.0
Discount (%)	2651.0	35.403621	35.162792	0.0	10.0	25.0
Revenue	2457.0	101.290307	159.300390	-37.5	0.0	0.0

Categorical Columns

In [240]:
```
sales_data.describe(include='object').T  # If you're using
current versions of pandas use 'str' instead of 'object'
```

Out[240]:

	count	unique	top	freq
Date	3100	104	2020-05-20	68
Product Category	3100	6	Protein	555
Product Name	3100	8	Slim F@st	409
Price	2457	4	12.50 USD	642
Platform	3100	6	Instore	537
Location	3100	8	L.A.	421

__Inspecting the columns in the dataset__

In [241]:
```
sales_data.columns
```

Out[241]:
```
Index(['Date', 'Product Category', 'Product Name', 'Units
Sold',
       'Units Returned', 'Price', 'Discount (%)', 'Revenue',
'Platform',
       'Location'],
      dtype='object')
```

__Inspecting data types in the dataset__

In [242]:
```
sales_data.dtypes
```

Out[242]:
```
Date                object
Product Category    object
Product Name        object
Units Sold         float64
Units Returned     float64
Price               object
Discount (%)       float64
Revenue            float64
Platform            object
Location            object
dtype: object
```

__Inspecting missing values__

In [243]:
```
sales_data.isnull().sum()
```

Out[243]:
```
Date                  0
Product Category      0
Product Name          0
Units Sold         1549
Units Returned      605
Price               643
Discount (%)        449
Revenue             643
Platform              0
Location              0
dtype: int64
```

STEP 4: Data Cleaning

4.1 Standardize Column names
Column names should be clean, readable, and consistent.

Why Standardisation Matters
- Prevents Code Errors: Programming languages like Python can fail if column names contain hidden spaces, trailing whitespaces, or special characters.
- Simplifies Merging: Joining datasets requires matching key columns; different names (e.g., user_id vs UserID) break automated joins.
- Speeds Up Queries: Consistent names allow you to write reusable code, macros, and templates without rewriting scripts for every new file.
- Improves Tool Automation: Modern BI tools (like Tableau or Power BI) and AI models automatically detect relationships when names match across tables.
- Enhances Readability: Clean, uniform names make it easy for teammates to understand the schema without reading data dictionaries.

Common Standardisation Practices
- Use lowercase letters exclusively (customer_id instead of CustomerID).
- Replace spaces with underscores (order_date instead of Order Date).

In [244]:
```
# Standardizing the sales column names
# 1. Strip any preceding spaces
sales_data.columns = sales_data.columns.str.strip()

# 3. Set the column names to lowercase
sales_data.columns = sales_data.columns.str.lower()



# 4. Remove any spaces between words and replace with underscores
sales_data.columns = sales_data.columns.str.replace(" ", "_")
```

In [245]:
```
# 2. Remove the percentage sign on the 'Discount (%)' and update
to discount percentage

sales_data = sales_data.rename(columns =
{"discount_(%)":"discount_percentage"})
```

In [246]:
```
sales_data.columns
```

Out[246]:
```
Index(['date', 'product_category', 'product_name',
       'units_sold',
              'units_returned', 'price', 'discount_percentage',
       'revenue', 'platform',
              'location'],
             dtype='object')
```

__Note:__ Instead of writing individual conversion, you can use the code below:
  `python
sales_data.columns = sales_data.columns.str.strip().str.lower().str.replace(' ', '_').str.replace('_(%)', '_percent', regex=False).str.replace('(%)', 'percent', regex=False)
  `

 `regex=False`
- RegEx (short for Regular Expression) is a sequence of special characters that forms a search pattern
- Setting regex=False tells the search or filter function to treat the search term as a literal string rather than a complex pattern. Without this, special characters (like . or *) are treated as active RegEx commands, which can cause unexpected errors.

4.2 Standardise data types

Common data types:
- Text/Object = names, categories, locations
- Integer = whole numbers
- Float = decimal numbers
- Date/Datetime = dates and times
- Boolean = True/False values

Common problems:

- Dates stored as text
- Numbers stored as text
- Currency values stored with symbols
- Percentages stored as text
- Yes/No values stored inconsistently

Date column

In [247]:
```
# Check the unique date formats
sales_data["date"].unique()

# This is important before converting the dates column into date
data type as it gives you a clear overview of how your dates
column looks like.
# Converting before checking the formats might cause errors or
removal of existing dates
```

Out[247]:
```
array(['2020-10-07', '2020-09-16', '2020-11-11', '2020-11-
04',
       '2020-06-03', '2020-09-09', '2020-06-17', '2020-09-
23',
       '2020-10-28', '2020-04-01', '2020-11-25', '2020-02-
19',
       '2020-09-02', '22/07/2020', '05/08/2020', '2020-01-
29',
       '08/04/2020', '2020-11-18', '2020-04-15', '2020-07-
15',
       '2020-07-29', '2020-07-08', '23/09/2020', '2020-07-
01',
       '2020-10-21', '2020-04-22', '2020-01-01',
'26/02/2020',
       '2020-09-30', '2020-12-23', '2020-04-08', '2020-05-
27',
       '2020-03-18', '2020-06-10', '2020-05-20', '2020-06-
24',
       '2020-07-22', '2020-02-05', '2020-10-14', '2020-03-
11',
       '2020-04-29', '2020-03-04', '01/07/2020', '2020-05-
13',
       '2020-02-12', '27/05/2020', '02/09/2020',
'06/05/2020',
       '2020-12-02', '2020-08-05', '2020-05-06',
'07/10/2020',
       '2020-01-08', '29/04/2020', '2020-08-26', '2020-08-
19',
       '30/09/2020', '2020-12-16', '2020-12-09', '2020-03-
25',
       '2020-01-15', '22/04/2020', '12/02/2020',
'19/02/2020',
       '22/01/2020', '2020-08-12', '18/11/2020',
'25/03/2020',
       '15/04/2020', '21/10/2020', '2020-01-22',
'16/09/2020',
       '02/12/2020', '01/04/2020', '14/10/2020',
'01/01/2020',
       '17/06/2020', '26/08/2020', '20/05/2020',
'24/06/2020',
       '15/01/2020', '05/02/2020', '15/07/2020', '2020-02-
26',
       '19/08/2020', '13/05/2020', '09/12/2020',
'09/09/2020',
       '16/12/2020', '29/01/2020', '11/11/2020',
'03/06/2020',
       '18/03/2020', '25/11/2020', '10/06/2020',
```

```
       '12/08/2020',
              '04/11/2020', '08/01/2020', '04/03/2020',
       '29/07/2020',
              '11/03/2020', '28/10/2020', '23/12/2020',
       '08/07/2020'],
             dtype=object)
```

In [248]:
```
# Convert the date column to a datetime format
sales_data["date"] = pd.to_datetime(sales_data["date"],
format='mixed', errors='coerce')
```

In [249]:
```
sales_data.dtypes
```

Out[249]:
```
date                  datetime64[ns]
product_category              object
product_name                  object
units_sold                   float64
units_returned               float64
price                         object
discount_percentage          float64
revenue                      float64
platform                      object
location                      object
dtype: object
```

In [250]:
```
# Check the unique entries in the price column
sales_data['price'].unique()
```

Out[250]:
```
array(['$10.99', nan, 15, '12.50 USD', 9.99], dtype=object)
```

In [251]:
```
# Dropping the dollar sign and the USD str
sales_data["price"] =
sales_data["price"].astype(str).str.replace(r'[^0-9.]', '',
regex=True)

# Convert the price column to numeric datatype
sales_data["price"] = pd.to_numeric(sales_data["price"],
errors='coerce')
```

- `.astype(str)` : Converts the entire column into text (strings) so that string-specific manipulation methods can be used.
- `.str.replace(...)` : Searches the text for specific patterns and replaces them.
- `r'[^0-9.]'` : This is a regular expression (regex).The ^ inside the brackets means "NOT".
- It translates to: _"Find anything that is not a number (0-9) and not a decimal point (.)"_.
- `''` : This replaces all the matched unwanted characters with an empty string (meaning they are deleted).,
- `regex=True` : Tells the replace function to use regex matching rather than looking for an exact text match.

In [252]:
```
sales_data.dtypes
```

Out[252]:
```
date                  datetime64[ns]
product_category              object
product_name                  object
units_sold                   float64
units_returned               float64
price                        float64
discount_percentage          float64
revenue                      float64
platform                      object
location                      object
dtype: object
```

units_returned column

There are negative entries in the units_returns which in reality there's no negative good

In [267]:
```
sales_data["units_returned"] = sales_data["units_returned"].abs()
```

In [268]:
```
sales_data.head()
```

Out[268]:

	date	product_category	product_name	units_sold	units_r
0	2020-10-07	Sleep Aids	ImmunoPro	NaN	1.0
1	2020-09-16	Vitamins	MuscleMax	NaN	1.0
2	2020-11-11	Immunity Boosters	SlimFast	15.0	1.0
3	2020-11-04	Energy Drinks	PowerFuel	14.0	0.0
4	2020-06-03	Vitamins	SuperVita	NaN	NaN

4.3 Check for Duplicates and drop them if any

In [253]:
```
# Check number of duplicates
print(f"Number of duplicate rows:
{sales_data.duplicated().sum()}")
```

```
Number of duplicate rows: 100
```

__Drop duplicated rows__

In [254]:
```
sales_data = sales_data.drop_duplicates()
print(f"Number of duplicate rows after:
{sales_data.duplicated().sum()}")
```

```
Number of duplicate rows after: 0
```

4.4 Handling Categorical columns entries

In [255]:
```
# Listing categorical columns
categorical_cols = sales_data.select_dtypes(include=
["object"]).columns.to_list()

categorical_cols

for column in categorical_cols:
    print("=" * 70)

    print(f"Unique categories in '{column}'")

    print(sales_data[column].unique())
```

```
======================================================================
=========
Unique categories in 'product_category'
['Sleep Aids' 'Vitamins' 'Immunity Boosters' 'Energy Drinks'
'Weight Loss'
 'Protein']
======================================================================
=========
Unique categories in 'product_name'
['ImmunoPro' 'MuscleMax' 'Slim F@st' 'PowerFuel' 'SuperV!ta'
'SuperVita'
 'DreamRest' 'SlimFast']
======================================================================
=========
Unique categories in 'platform'
['Instore' 'Mob App' 'Online' 'Mobile App' 'In-Store'
'onlien']
======================================================================
=========
Unique categories in 'location'
['Los Angeles' 'Houston' 'Chicago' 'new york' 'Phoenix' 'New
York'
 'Chi-Town' 'L.A.']
```

__Clean product_name, platform, and location__

product_name
Clean the product_name column and unify the product names i.e 'Slim F@st' should be 'SlimFast', 'SuperV!ta' should be 'SuperVita'

In [259]:
```
sales_data["product_name"] =
sales_data["product_name"].replace({'Slim F@st' : 'SlimFast',

'SuperV!ta' : 'SuperVita'})
```

platform
Clean and unify the platform column categories i.e 'In-Store' should be 'Instore', 'onlien' should be 'Online', 'Mob App' should be 'Mobile App'

In [262]:
```
sales_data['platform'] = sales_data['platform'].replace({'In-
Store':'Instore',
                                                         'Mob App'
: 'Mobile App',
                                                         'onlien'
: 'Online'})
```

location
Clean and unify the location column categories i.e 'L.A.' should be 'Los Angeles', 'Chi-Town' should be 'Chicago', 'new york' should be 'New York'

In [264]:
```
sales_data['location'] = sales_data['location'].replace({'Chi-
Town':'Chicago',
                                                         'new
york' : 'New York',
                                                         'L.A.' :
'Los Angeles'})
```

__Confirm implementation of changes__

In [266]:
```
for column in categorical_cols:
    print("=" * 70)

    print(f"Unique categories in '{column}'")

    print(sales_data[column].unique())
```

```
============================================================
=========
Unique categories in 'product_category'
['Sleep Aids' 'Vitamins' 'Immunity Boosters' 'Energy Drinks'
'Weight Loss'
 'Protein']
============================================================
=========
Unique categories in 'product_name'
['ImmunoPro' 'MuscleMax' 'SlimFast' 'PowerFuel' 'SuperVita'
'DreamRest']
============================================================
=========
Unique categories in 'platform'
['Instore' 'Mobile App' 'Online']
============================================================
=========
Unique categories in 'location'
['Los Angeles' 'Houston' 'Chicago' 'New York' 'Phoenix']
```

4.5 Handling Missing Values

There is no single correct method. It depends on the column and the business context.

In [269]:
```
sales_data.head()
```

Out[269]:

	date	product_category	product_name	units_sold	units_r
0	2020-10-07	Sleep Aids	ImmunoPro	NaN	1.0
1	2020-09-16	Vitamins	MuscleMax	NaN	1.0
2	2020-11-11	Immunity Boosters	SlimFast	15.0	1.0
3	2020-11-04	Energy Drinks	PowerFuel	14.0	0.0
4	2020-06-03	Vitamins	SuperVita	NaN	NaN

In [ ]: